

Advanced Level (Laboratorios)

Módulo. Arquitectura Limpia

Laboratorio:

Reestructurar Orders a arquitectura limpia; introducir UoW y Presenter; evento OrderCreated manejado en aplicación

Estructura del Módulo 1 (src/advanced_level/modulo01/)

```
src/advanced_level/modulo01/
├── domain/                    # Capa 1: Entidades, Reglas de Negocio y Eventos
│   ├── events.py             # Contrato base de eventos de dominio
│   ├── exceptions.py         # Excepciones puras de dominio
│   └── order.py              # Agregado Order, OrderItem y eventos asociados
├── application/              # Capa 2: Casos de Uso y Unidades de Trabajo
│   ├── dtos.py               # DTOs de entrada/salida
│   ├── unit_of_work.py       # Puerto/Interfaz del Unit of Work
│   ├── event_bus.py          # Bus de eventos y manejadores
│   └── use_cases.py           # Caso de uso PlaceOrder
├── adapters/                 # Capa 3: Controladores, Presentadores y Gateways
│   ├── repositories.py       # Gateways de lectura/escritura
│   └── presenters.py         # Transformadores a respuestas estructuradas
└── infrastructure/           # Capa 4: Frameworks, BD y FastAPI
    ├── database.py           # SQLAlchemy engine, session maker y modelos
    ├── uow_impl.py           # Implementación concreta del UoW con SQLAlchemy
    └── api.py                 # Router y controladores de FastAPI
```

1. Capa de Dominio – Eventos, Excepciones y Agregado Order

En esta capa no se importa absolutamente nada externo (ni SQLAlchemy, ni FastAPI ni Pydantic)

- Eventos de Dominio (src/advanced_level/modulo01/domain/events.py)

```
import uuid
from dataclasses import dataclass, field
from datetime import UTC, datetime

@dataclass(frozen=True)
class DomainEvent:
    """Clase base para todos los eventos de dominio del sistema."""

    event_id: str = field(default_factory=lambda: str(uuid.uuid4()))
```

```

    occurred_on: datetime = field(
        default_factory=lambda: datetime.now(UTC)
    )

@dataclass(frozen=True)
class OrderPlacedEvent(DomainEvent):
    """Evento emitido cuando un pedido es creado y registrado con
    éxito."""

    order_id: str = ""
    customer_id: str = ""
    total_amount: float = 0.0

```

- Excepciones del Dominio
(src/advance_level/modulo01/domain/exceptions.py)

```

class DomainError(Exception):
    """Excepción base para todos los errores de dominio."""

class InvalidOrderError(DomainError):
    """Se lanza cuando un pedido viola alguna regla de negocio."""

class OutOfStockError(DomainError):
    """Se lanza cuando un producto no tiene suficiente stock
    disponible."""

```

- Entidad de Dominio / Agregado Order
(src/advanced_level/modulo01/domain/order.py)

```

import uuid
from dataclasses import dataclass, field

from ...modulo01.domain.events import (
    DomainEvent,

```

```

        OrderPlacedEvent,
    )
from ...modulo01.domain.exceptions import InvalidOrderError

@dataclass
class OrderItem:
    product_id: str
    quantity: int
    unit_price: float

    def __post_init__(self) -> None:
        if self.quantity <= 0:
            raise InvalidOrderError("La cantidad del producto debe ser
mayor a 0.")
        if self.unit_price <= 0.0:
            raise InvalidOrderError("El precio unitario debe ser mayor
a 0.")

    @property
    def subtotal(self) -> float:
        return self.quantity * self.unit_price

@dataclass
class Order:
    order_id: str
    customer_id: str
    items: list[OrderItem] = field(default_factory=list)
    status: str = "PENDING"
    events: list[DomainEvent] = field(default_factory=list)

    def __post_init__(self) -> None:
        if not self.customer_id.strip():
            raise InvalidOrderError("El cliente no puede ser un valor
vacío.")
        if not self.items:
            raise InvalidOrderError("Un pedido debe contener al menos
un ítem.")

    @classmethod

```

```

    def create(cls, customer_id: str, items: list[OrderItem]) ->
"Order":
    order = cls(
        order_id=str(uuid.uuid4()),
        customer_id=customer_id,
        items=items,
        status="PLACED",
    )
    # Registramos el evento de dominio tras crearse exitosamente
    order.events.append(
        OrderPlacedEvent(
            order_id=order.order_id,
            customer_id=order.customer_id,
            total_amount=order.total_amount,
        )
    )
    return order

@property
def total_amount(self) -> float:
    return sum(item.subtotal for item in self.items)

```

2. Capa de Aplicación - UoW, Event Bus y Casos de Uo

- DTOs de Aplicación (src/advanced_level/modulo01/application/dtos.py)

```

from dataclasses import dataclass, field

@dataclass(frozen=True)
class OrderItemDTO:
    product_id: str
    quantity: int
    unit_price: float

@dataclass(frozen=True)
class PlaceOrderDTO:
    customer_id: str
    items: list[OrderItemDTO]

```

```
@dataclass(frozen=True)
class OrderResponseDTO:
    order_id: str
    customer_id: str
    total_amount: float
    status: str
    message: str = field(default="Pedido registrado exitosamente.")
```

- Puerto del Repositorio
(src/advanced_level/modulo01/adapters/repositories.py)

```
from typing import Protocol

from ...modulo01.domain.order import Order

class OrderRepositoryPort(Protocol):
    """Interfaz Gateway para la persistencia de órdenes."""

    def add(self, order: Order) -> None:
        ...

    def get_by_id(self, order_id: str) -> Order | None:
        ...
```

- Puerto del Unit of Work
(src/advanced_level/modulo01/application/unit_of_work.py)

El Unit of Work maneja el contexto de la transacción de forma atómica y expone las colecciones de repositorios necesarias.

```
from typing import Protocol, Self

from ...modulo01.adapters.repositories import OrderRepositoryPort

class UnitOfWorkPort(Protocol):
    """Interfaz para gestionar la transacción atómica (Unit of Work)."""
```

```

orders: OrderRepositoryPort

def __enter__(self) -> Self:
    ...

def __exit__(self, exc_type: type | None, exc_val: Exception |
None, traceback: object) -> None:
    ...

def commit(self) -> None:
    ...

def rollback(self) -> None:
    ...

```

- Event Bus (src/advances_level/modulo01/application/event_bus.py)

```

from collections import defaultdict
from collections.abc import Callable
from typing import Any, TypeVar

from ...modulo01.domain.events import DomainEvent

E = TypeVar("E", bound=DomainEvent)
EventHandler = Callable[[E], None]

class EventBus:
    """Manejador/Despachador central de eventos de dominio."""

    def __init__(self) -> None:
        self._handlers: dict[type[DomainEvent], list[Callable[[Any],
None]]] = defaultdict(list) # type: ignore[name-defined]

    def subscribe(self, event_type: type[E], handler: EventHandler[E])
-> None:
        self._handlers[event_type].append(handler) # type: ignore[arg-
type]

    def publish(self, events: list[DomainEvent]) -> None:

```

```

for event in events:
    event_type = type(event)
    handlers = self._handlers.get(event_type, [])
    for handler in handlers:
        handler(event)

```

- Caso de Uso PlaceOrderUseCase
(src/advancedlevel/modulo01/application/use_cases.py)

El caso de uso coordina la transacción mediante el UoW y desata la publicación de los eventos de dominio una vez confirmado el commit:

```

from ...modulo01.application.dtos import OrderResponseDTO,
PlaceOrderDTO
from ...modulo01.application.event_bus import EventBus
from ...modulo01.application.unit_of_work import UnitOfWorkPort
from ...modulo01.domain.order import Order, OrderItem

class PlaceOrderUseCase:
    """Caso de Uso: Registrar una orden de compra en el sistema."""

    def __init__(self, uow: UnitOfWorkPort, event_bus: EventBus) ->
None:
        self.uow = uow
        self.event_bus = event_bus

    def execute(self, dto: PlaceOrderDTO) -> OrderResponseDTO:
        # 1. Crear los objetos de valor/entidades del dominio
        items = [
            OrderItem(
                product_id=item_dto.product_id,
                quantity=item_dto.quantity,
                unit_price=item_dto.unit_price,
            )
            for item_dto in dto.items
        ]

        order = Order.create(customer_id=dto.customer_id, items=items)

        # 2. Ejecutar la persistencia atómica con Unit of Work
        with self.uow:
            self.uow.orders.add(order)

```

```

        self.uow.commit()

        # 3. Despachar eventos de dominio acumulados en la entidad tras
        el commit exitoso
        self.event_bus.publish(order.events)
        order.events.clear()

        # 4. Retornar DTO de respuesta
        return OrderResponseDTO(
            order_id=order.order_id,
            customer_id=order.customer_id,
            total_amount=order.total_amount,
            status=order.status,
        )

```

3. Infraestructura (src/advanced_level/modulo01/infrastructure)

- Mapeo de Base de Datos
(src/advanced_level/modulo01/infrastructure/databas.py)

```

from sqlalchemy import Column, Float, Integer, String
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass

class OrderModel(Base):
    __tablename__ = "orders"

    order_id = Column(String, primary_key=True)
    customer_id = Column(String, nullable=False)
    status = Column(String, nullable=False)

class OrderItemModel(Base):
    __tablename__ = "order_items"

    id = Column(Integer, primary_key=True, autoincrement=True)

```

```
order_id = Column(String, nullable=False)
product_id = Column(String, nullable=False)
quantity = Column(Integer, nullable=False)
unit_price = Column(Float, nullable=False)
```

- Implementación del Repositorio
(src/advanced_level/modulo01/infrastructure/repository_impl.py)

```
from sqlalchemy.orm import Session

from ...modulo01.adapters.repositories import OrderRepositoryPort
from ...modulo01.domain.order import Order, OrderItem

class SQLAlchemyOrderRepository(OrderRepositoryPort):
    """Adaptador de persistencia usando una sesión activa de
    SQLAlchemy."""

    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, order: Order) -> None:
        from ...modulo01.infrastructure.database import (
            OrderItemModel,
            OrderModel,
        )

        order_record = OrderModel(
            order_id=order.order_id,
            customer_id=order.customer_id,
            status=order.status,
        )
        self.session.merge(order_record)

        for item in order.items:
            item_record = OrderItemModel(
                order_id=order.order_id,
                product_id=item.product_id,
                quantity=item.quantity,
                unit_price=item.unit_price,
            )
```

```

        self.session.add(item_record)

def get_by_id(self, order_id: str) -> Order | None:
    from ...modulo01.infrastructure.database import (
        OrderItemModel,
        OrderModel,
    )

    order_record = (
        self.session.query(OrderModel)
        .filter(OrderModel.order_id == order_id)
        .first()
    )
    if order_record is None:
        return None

    item_records = (
        self.session.query(OrderItemModel)
        .filter(OrderItemModel.order_id == order_id)
        .all()
    )

    items = [
        OrderItem(
            product_id=str(rec.product_id),
            quantity=int(rec.quantity),
            unit_price=float(rec.unit_price),
        )
        for rec in item_records
    ]

    return Order(
        order_id=str(order_record.order_id),
        customer_id=str(order_record.customer_id),
        items=items,
        status=str(order_record.status),
    )

```

- Implementación concreta de Unit of Work
(src/advanced_level/modulo01/infrastructure/uow_impl.py)

```
from typing import Self

from sqlalchemy.orm import Session, sessionmaker

from ...modulo01.application.unit_of_work import UnitOfWorkPort
from ...modulo01.infrastructure.repository_impl import (
    SQLAlchemyOrderRepository,
)

class SQLAlchemyUnitOfWork(UnitOfWorkPort):
    """Implementación de Unit of Work que coordina transacciones con
    SQLAlchemy."""

    def __init__(self, session_factory: sessionmaker[Session]) -> None:
        self.session_factory = session_factory
        self.session: Session | None = None

    def __enter__(self) -> Self:
        self.session = self.session_factory()
        self.orders = SQLAlchemyOrderRepository(self.session)
        return self

    def __exit__(
        self,
        exc_type: type | None,
        exc_val: Exception | None,
        traceback: object,
    ) -> None:
        if exc_type is not None:
            self.rollback()
        if self.session is not None:
            self.session.close()

    def commit(self) -> None:
        if self.session is not None:
            self.session.commit()
```

```
def rollback(self) -> None:
    if self.session is not None:
        self.session.rollback()
```

- Controlador y Router FaaS API
(src/advanced_level/modulo01/infrastructure/api.py)

```
from fastapi import APIRouter, HTTPException, status
from pydantic import BaseModel, ConfigDict, Field

from ...modulo01.application.dtos import (
    OrderItemDTO,
    PlaceOrderDTO,
)
from ...modulo01.application.event_bus import EventBus
from ...modulo01.application.use_cases import PlaceOrderUseCase
from ...modulo01.domain.exceptions import DomainError
from ...modulo01.infrastructure.uow_impl import SQLAlchemyUnitOfWork

class ItemRequestSchema(BaseModel):
    product_id: str = Field(..., min_length=1)
    quantity: int = Field(..., gt=0)
    unit_price: float = Field(..., gt=0.0)

class PlaceOrderRequestSchema(BaseModel):
    customer_id: str = Field(..., min_length=1)
    items: list[ItemRequestSchema] = Field(..., min_items=1)

class OrderResponseSchema(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    order_id: str
    customer_id: str
    total_amount: float
    status: str
    message: str
```

```

def create_order_router(uow: SQLAlchemyUnitOfWork, event_bus: EventBus)
-> APIRouter:
    router = APIRouter(prefix="/orders", tags=["Orders"])

    @router.post("", response_model=OrderResponseSchema,
status_code=status.HTTP_201_CREATED)
    def place_order(payload: PlaceOrderRequestSchema) ->
OrderResponseSchema:
        use_case = PlaceOrderUseCase(uow=uow, event_bus=event_bus)

        items_dto = [
            OrderItemDTO(
                product_id=item.product_id,
                quantity=item.quantity,
                unit_price=item.unit_price,
            )
            for item in payload.items
        ]

        dto = PlaceOrderDTO(customer_id=payload.customer_id,
items=items_dto)

        try:
            result = use_case.execute(dto)
            return OrderResponseSchema(
                order_id=result.order_id,
                customer_id=result.customer_id,
                total_amount=result.total_amount,
                status=result.status,
                message=result.message,
            )
        except DomainError as err:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
detail=str(err)
            )

    return router

```

4. Pruebas (tests/test_modulo01.py)

```
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from src.modulo01.application.dtos import OrderItemDTO, PlaceOrderDTO
from src.modulo01.application.event_bus import EventBus
from src.modulo01.application.use_cases import PlaceOrderUseCase
from src.modulo01.domain.events import OrderPlacedEvent
from src.modulo01.domain.exceptions import InvalidOrderError
from src.modulo01.domain.order import Order, OrderItem
from src.modulo01.infrastructure.database import Base
from src.modulo01.infrastructure.uow_impl import SQLAlchemyUnitOfWork

# Fixtures para BBDD en Memoria, UoW y EventBus

@pytest.fixture
def uow():
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(engine)
    session_factory = sessionmaker(bind=engine)
    return SQLAlchemyUnitOfWork(session_factory=session_factory)

@pytest.fixture
def event_bus():
    return EventBus()

# 1. PRUEBAS DE DOMINIO Y EVENTOS

def test_order_creation_generates_domain_event():
    items = [OrderItem(product_id="PROD-1", quantity=2,
unit_price=50.0)]
    order = Order.create(customer_id="CUST-001", items=items)

    assert order.total_amount == 100.0
```

```

    assert len(order.events) == 1
    event = order.events[0]
    assert isinstance(event, OrderPlacedEvent)
    assert event.customer_id == "CUST-001"
    assert event.total_amount == 100.0

def test_invalid_order_item_quantity_raises_error():
    with pytest.raises(InvalidOrderError):
        OrderItem(product_id="PROD-1", quantity=0, unit_price=10.0)

# 2. PRUEBA INTEGRADA DEL CASO DE USO CON UOW Y EVENT BUS

def test_place_order_use_case_with_uow_and_events(uow, event_bus):
    handled_events: list[OrderPlacedEvent] = []

    # Suscribirse un handler ficticio al evento OrderPlacedEvent
    def on_order_placed(event: OrderPlacedEvent) -> None:
        handled_events.append(event)

    event_bus.subscribe(OrderPlacedEvent, on_order_placed)

    use_case = PlaceOrderUseCase(uow=uow, event_bus=event_bus)

    dto = PlaceOrderDTO(
        customer_id="CUST-999",
        items=[
            OrderItemDTO(product_id="LAPTOP", quantity=1,
unit_price=1200.0),
            OrderItemDTO(product_id="MOUSE", quantity=2,
unit_price=25.0),
        ],
    )

    response = use_case.execute(dto)

    # 1. Verificar la respuesta del DTO
    assert response.order_id is not None
    assert response.customer_id == "CUST-999"
    assert response.total_amount == 1250.0

```

```

assert response.status == "PLACED"

# 2. Verificar la persistencia mediante UoW
with uow:
    retrieved_order = uow.orders.get_by_id(response.order_id)
    assert retrieved_order is not None
    assert retrieved_order.customer_id == "CUST-999"
    assert retrieved_order.total_amount == 1250.0

# 3. Verificar que el evento fue despachado al EventBus
assert len(handled_events) == 1
assert handled_events[0].order_id == response.order_id
assert handled_events[0].total_amount == 1250.0

```

En este laboratorio vamos a consolidar y validar la arquitectura asegurando que la separación entre Dominio, Aplicación e Infraestructura cumpla estrictamente con la regla de dependencias, introduciendo de forma explícita el Presenter para desacoplar el formato de respuesta del framework web y conectando el manejador del evento de dominio OrderCreated.

Plaintext

```

src/modulo01/
├── domain/
│   ├── exceptions.py
│   ├── events.py
│   ├── models.py
│   └── repository.py
├── application/
│   ├── dtos.py
│   ├── event_bus.py
│   ├── handlers.py      <-- Manejador del evento OrderCreated
│   ├── presenters.py    <-- Puerto / Interface para el Presenter
│   ├── unit_of_work.py
│   └── use_cases.py
└── infrastructure/
    ├── api.py
    ├── presenters_impl.py <-- Implementación concreta del Presenter
    ├── repository_impl.py
    └── uow_impl.py

```

1. Definir el Presente y el Event Handler en Aplicación

A. Puerto e Implementación del Presenter (presenters.py y presenters_impl.py)

El Presenter se encarga de transformar el resultado del caso de uso o DTOs en una estructura óptima para la capa de entrega (API/HTTP), evitando que el Use Case conozca los esquemas de FastAPI.

```
from typing import Generic, Protocol, TypeVar

from ...modulo01.application.dtos import OrderResponseDTO

T = TypeVar("T", covariant=True)

class OrderPresenterPort(Protocol, Generic[T]):
    """Puerto para transformar el resultado del caso de uso."""

    def present_order_created(self, dto: OrderResponseDTO) -> T:
        ...
```

- Src/modulo01/infrastructure/presenters_impl.py:

```
from typing import Any

from ...modulo01.application.dtos import OrderResponseDTO
from ...modulo01.application.presenters import OrderPresenterPort

class JsonOrderPresenter(OrderPresenterPort[dict[str, Any]]):
    """Implementación que genera un diccionario adecuado para
    respuestas JSON en API."""

    def present_order_created(self, dto: OrderResponseDTO) -> dict[str, Any]:
        return {
            "data": {
                "order_id": dto.order_id,
                "customer_id": dto.customer_id,
```

```

        "total_amount": dto.total_amount,
        "status": dto.status,
    },
    "meta": {
        "message": dto.message,
        "code": "ORDER_CREATED_SUCCESSFULLY",
    },
}

```

B. Manejador del Evento OrderCreated (handlers.py)

Aseguramos que cuando el dominio emita el evento OrderCreated, la capa de aplicación reaccione desacoplada (por ejemplo, simulando el envío de una notificación de pedido)

```

from ...modulo01.domain.events import OrderPlacedEvent

class OrderNotificationsHandler:
    """Suscriptor/Manejador de eventos que reacciona a
    OrderPlacedEvent."""

    def handle_order_created(self, event: OrderPlacedEvent) -> None:
        print(
            f"[EVENT HANDLER] Notificación enviada para el pedido
            {event.order_id} "
            f"del cliente {event.customer_id} por
            ${event.total_amount:.2f}"
        )

```

2. Conectar Presenter y Handler en la API (api.py)

Actualizamos la ruta de FastAPI en src/modulo01/infrastructure/api.py para:

- Registrar el OrderNotificationHandler en el EventBus.
- Delegar la respuesta HTTP al JsonOrderPresenter.

```

from typing import Any

from fastapi import APIRouter, HTTPException, status
from pydantic import BaseModel, Field

from ...modulo01.application.dtos import OrderItemDTO, PlaceOrderDTO

```

```

from ...modulo01.application.event_bus import EventBus
from ...modulo01.application.handlers import OrderNotificationsHandler
from ...modulo01.application.use_cases import PlaceOrderUseCase
from ...modulo01.domain.events import OrderPlacedEvent
from ...modulo01.domain.exceptions import DomainError
from ...modulo01.infrastructure.presenters_impl import
JsonOrderPresenter
from ...modulo01.infrastructure.uow_impl import SQLAlchemyUnitOfWork

class ItemRequestSchema(BaseModel):
    product_id: str = Field(min_length=1)
    quantity: int = Field(gt=0)
    unit_price: float = Field(gt=0.0)

class PlaceOrderRequestSchema(BaseModel):
    customer_id: str = Field(min_length=1)
    items: list[ItemRequestSchema] = Field(min_length=1)

def create_order_router(uow: SQLAlchemyUnitOfWork, event_bus: EventBus)
-> APIRouter:
    router = APIRouter(prefix="/orders", tags=["Orders"])

    # 1. Suscribirse el manejador de eventos
    notification_handler = OrderNotificationsHandler()
    event_bus.subscribe(
        OrderPlacedEvent, notification_handler.handle_order_created
    )

    presenter = JsonOrderPresenter()

    @router.post("", status_code=status.HTTP_201_CREATED)
    def place_order(payload: PlaceOrderRequestSchema) -> dict[str,
Any]:
        use_case = PlaceOrderUseCase(uow=uow, event_bus=event_bus)

        items_dto = [
            OrderItemDTO(
                product_id=item.product_id,
                quantity=item.quantity,

```

```

        unit_price=item.unit_price,
    )
    for item in payload.items
]

    dto = PlaceOrderDTO(customer_id=payload.customer_id,
items=items_dto)

    try:
        result_dto = use_case.execute(dto)
        # 2. El Presenter da el formato final de respuesta
        return presenter.present_order_created(result_dto)
    except DomainError as err:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
detail=str(err)
        )

    return router

```

3. Test de Integración para validar el laboratorio

Agregamos una prueba en tests/test_modulo01.py que valide:

- La ejecución atómica vía Unit of Work.
- La captura y ejecución del evento OrderCreatedEvent por Handler.
- La transformación de la salida mediante el Presenter.

```

import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from src.modulo01.application.dtos import OrderItemDTO, PlaceOrderDTO
from src.modulo01.application.event_bus import EventBus
from src.modulo01.application.use_cases import PlaceOrderUseCase
from src.modulo01.domain.events import OrderPlacedEvent
from src.modulo01.infrastructure.database import Base
from src.modulo01.infrastructure.presenters_impl import
JsonOrderPresenter
from src.modulo01.infrastructure.uow_impl import SQLAlchemyUnitOfWork

```

```

@pytest.fixture
def session_factory():
    engine = create_engine("sqlite:///memory:")
    Base.metadata.create_all(engine)
    return sessionmaker(bind=engine)

def test_laboratorio_uow_presenter_and_events(session_factory):
    # Setup
    uow = SQLAlchemyUnitOfWork(session_factory)
    event_bus = EventBus()

    events_captured: list[OrderPlacedEvent] = []

    def spy_event_handler(event: OrderPlacedEvent) -> None:
        events_captured.append(event)

    event_bus.subscribe(OrderPlacedEvent, spy_event_handler)

    use_case = PlaceOrderUseCase(uow=uow, event_bus=event_bus)
    presenter = JsonOrderPresenter()

    # Act
    dto = PlaceOrderDTO(
        customer_id="CUST-LAB-100",
        items=[OrderItemDTO(product_id="PROD-01", quantity=2,
unit_price=150.0)],
    )
    result_dto = use_case.execute(dto)
    response_payload = presenter.present_order_created(result_dto)

    # Assert 1: Presenter output
    assert response_payload["data"]["customer_id"] == "CUST-LAB-100"
    assert response_payload["data"]["total_amount"] == 300.0
    assert response_payload["meta"]["code"] ==
"ORDER_CREATED_SUCCESSFULLY"

    # Assert 2: Event published and handled
    assert len(events_captured) == 1
    assert events_captured[0].customer_id == "CUST-LAB-100"

```

```

assert events_captured[0].total_amount == 300.0

# Assert 3: Persisted via UoW
with uow:
    assert uow.orders is not None
    saved_order = uow.orders.get_by_id(result_dto.order_id)
    assert saved_order is not None
    assert saved_order.customer_id == "CUST-LAB-100"

```

4. Ejecución de las comprobaciones de calidad y pruebas:

```

(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/advanced_level (main)
• $ poetry run ruff check .
All checks passed!
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/advanced_level (main)
• $ poetry run ruff check . --fix
All checks passed!
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/advanced_level (main)
• $ poetry run ruff format
6 files reformatted, 13 files left unchanged
(intermediate-level-py3.13)

```

```

$ OMP_NUM_THREADS=1 poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\advanced_level
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-5.0.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 1 item

tests\test_modulo01.py . [100%]

===== 1 passed in 1.04s =====
(intermediate-level-py3.13)

```

```

$ OMP_NUM_THREADS=1 poetry run pytest --cov=src --cov-report=term-missing
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 1 item

tests\test_modulo01.py . [100%]

----- coverage: platform win32, python 3.13.15-final-0 -----
Name                                                    Stmts  Miss  Cover   Missing
-----
src\advanced_level\__init__.py                          0      0  100%
src\modulo01\adapters\repositories.py                    3      0  100%
src\modulo01\application\dtos.py                         17      0  100%
src\modulo01\application\event_bus.py                   17      0  100%
src\modulo01\application\presenters.py                   4      0  100%
src\modulo01\application\unit_of_work.py                 4      0  100%
src\modulo01\application\use_cases.py                   17      0  100%
src\modulo01\domain\events.py                           12      0  100%
src\modulo01\domain\exceptions.py                       3      0  100%
src\modulo01\domain\order.py                            37      4   89%  19, 21, 38, 40
src\modulo01\infrastructure\database.py                 16      0  100%
src\modulo01\infrastructure\presenters_impl.py           6      0  100%
src\modulo01\infrastructure\repository_impl.py          21      1   95%  47
src\modulo01\infrastructure\uow_impl.py                 23      3   87%  30, 39-40
TOTAL                                                    180      8   96%

===== 1 passed in 0.89s =====

```

Módulo. Empaquetado, distribución y CI/CD

Construcción de Dockerfile multi-etapa (multistage) diseñado para el proyecto con FastAPI, Pydantic V2 y SQLAlchemy.

Este archivo aplica 5 buenas prácticas industriales:

- Multistage Build: Separa el entorno de construcción (builder) del de ejecución (runtime) para no incluir herramientas de compilación ni librerías de desarrollo en la imagen final.
- Cache de capas por dependencias: Exporta e instala los paquetes antes de copiar el código fuente para acelerar builds subsiguientes.
- Seguridad (Non-root user): Crea un usuario sin privilegios (appuser) para evitar vulnerabilidades de ejecución como root.
- Optimización de Python: Configura variables de entorno (PYTHONDONTWRITEBYTECODE, PYTHONUNBUFFERED) para logs limpios y menor huella de disco.
- Healthcheck: Permite a orquestadores (Docker, Compose, Azure Container Apps, Kubernetes) monitorear la salud del contenedor.

1. El Dockerfile Multi-etapa:

```
# STAGE 1: Builder
```

```
FROM python:3.13-slim AS builder

# Evitar escritura de archivos .pyc y forzar stdout/stderr sin buffer
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    POETRY_VERSION=1.8.3 \
    POETRY_HOME="/opt/poetry" \
    POETRY_NO_INTERACTION=1 \
    POETRY_VIRTUALENVS_IN_PROJECT=1 \
    POETRY_VIRTUALENVS_CREATE=1

# Instalar dependencias del sistema mínimas para compilar
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Instalar Poetry
RUN curl -sSL https://install.python-poetry.org | python3 -
ENV PATH="$POETRY_HOME/bin:$PATH"

WORKDIR /app

# Copiar archivos de definición de dependencias
COPY pyproject.toml poetry.lock* ./

# Instalar dependencias del proyecto (solo de producción)
RUN poetry install --no-root --only main

# STAGE 2: Runtime

FROM python:3.13-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PATH="/app/.venv/bin:$PATH" \
    PORT=8000

# Instalar dependencias en tiempo de ejecución (ej. curl para el healthcheck)
RUN apt-get update && apt-get install -y --no-install-recommends \
```

```

curl \
&& rm -rf /var/lib/apt/lists/*

# Principio de menor privilegio: Crear usuario no-root
RUN groupadd -g 10001 appgroup && \
    useradd -u 10001 -g appgroup -s /bin/sh -m appuser

WORKDIR /app

# Copiar el entorno virtual con todas las dependencias desde el builder
COPY --from=builder --chown=appuser:appgroup /app/.venv /app/.venv

# Copiar el código fuente de la aplicación desde src/
COPY --chown=appuser:appgroup ./src /app/src

# Cambiar al usuario no administrativo
USER appuser

EXPOSE 8000

# Healthcheck para monitorear el estado del contenedor
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:${PORT}/health || exit 1

# Comando por defecto para iniciar la aplicación FastAPI
CMD ["uvicorn", "src.modulo01.infraestructure.api:app", "--host",
"0.0.0.0", "--port", "8000"]

```

2. Archivo .dockerignore

Es crucial no incluir artefactos locales, entornos virtuales, caches ni carpetas de git dentro del contexto del build de Docker.

```

.git
.gitignore
.venv
__pycache__
*.pyc
*.pyo
*.pyd

```

```
.pytest_cache
.mypy_cache
.ruff_cache
.coverage
htmlcov
dist
build
*.egg-info
.env
.vscode
.idea
```

3. Modificación rápida en la API para el Healthcheck

Para que el HEALTHCHECK del Dockerfile funcione correctamente, agrega un endpoint /healt en la instancia principal de FastAPI o en tu router de infraestructura.

```
@router.get("/healt", tags=["Healt"])
def health_check() -> dict[str, str]:
    return {"status": "ok"}
```

- Comprobar la construcción y ejecución local

```

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
$ docker build -t modulo01-clean-arch:v1 .
=> => transferring dockerfile: 2.11kB 0.0s
=> [internal] load metadata for docker.io/library/python:3.13-s 1.1s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 207B 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 4.01kB 0.0s
=> [builder 1/6] FROM docker.io/library/python:3.13-slim@sha256 0.1s
=> => resolve docker.io/library/python:3.13-slim@sha256:7c61056 0.1s
=> CACHED [runtime 2/6] RUN apt-get update && apt-get install - 0.0s
=> CACHED [runtime 3/6] RUN groupadd -g 10001 appgroup && u 0.0s
=> CACHED [runtime 4/6] WORKDIR /app 0.0s
=> CACHED [builder 2/6] RUN apt-get update && apt-get install - 0.0s
=> CACHED [builder 3/6] RUN curl -sSL https://install.python-po 0.0s
=> CACHED [builder 4/6] WORKDIR /app 0.0s
=> CACHED [builder 5/6] COPY pyproject.toml poetry.lock* ./ 0.0s
=> CACHED [builder 6/6] RUN poetry install --no-root --only mai 0.0s
=> CACHED [runtime 5/6] COPY --from=builder --chown=appuser:app 0.0s
=> [runtime 6/6] COPY --chown=appuser:appgroup ./src /app/src 0.1s
=> exporting to image 0.7s
=> => exporting layers 0.1s
=> => exporting manifest sha256:8252e8023d27797f3bdf36b0073da7a 0.0s
=> => exporting config sha256:495fd9254417914b87fc371ab013880b5 0.0s
=> => exporting attestation manifest sha256:a67077dbf65983a7310 0.1s
=> => exporting manifest list sha256:6bffb149223330b3b64470a7ce 0.1s
=> => naming to docker.io/library/modulo01-clean-arch:v1 0.0s
=> => unpacking to docker.io/library/modulo01-clean-arch:v1 0.2s
(intermediate-level-py3.13)

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
$ docker run -d --name order-service -p 8000:8000 modulo01-clean-arch:v1
1dd87a047750f44964c9d1ef261d777e671f7079d566c1c0747bfcd68322f1bf
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
$ docker logs orders-service
Error response from daemon: No such container: orders-service
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
$ docker logs order-service
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
$ docker exec order-service whoami
appuser
(intermediate-level-py3.13)

```

5. Monitorear el Workflow

CI/CD Pipeline - Order Service

Filter workflow runs

Help us improve GitHub Actions
Tell us how to make GitHub Actions work better for you with three quick questions. [Give feedback](#)

32 workflow runs

Event Status Branch Actor

fix(ci): Establecer el contexto en advanced_level para...
CI/CD Pipeline - Order Service #32: Commit 00ea772 pushed by juanrivera-A
1 hour ago
1m 47s

- Validamos los jobs

Confirmamos que ambos jobs se muestren en verde:

- Quality-and-test: Confirmara ruff (linter/formatter), mypy (tipos), pytest (pruebas) y la generación del .whl (guardado en los Artifacts de la carrera).
- Build-and-push-docker: Se activará inmediatamente después, autenticará con ghcr.io, construirá la imagen multi-satge y publicará la nueva versión.

CI/CD Pipeline - Order Service

fix(ci): Establecer el contexto en advanced_level para que COPY src/ ... #32

Re-run all jobs

Summary

All jobs

quality-and-test

build-and-push-docker

Run details

Usage

Workflow file

Triggered via push 1 hour ago

juanrivera-A pushed 00ea772 main

Status Success

Total duration 1m 47s

Artifacts 2

ci.yml
on: push

quality-and-test 29s

build-and-push-docker 1m 12s

- Confirmamos la publicación del contenedor:

capacitacion-python sha-00ea77283c4c95d836d4ace14cba3b03cef10eaa Public

Latest

Installation OS / Arch 2

[Learn more about packages](#)

Install from the command line

```
$ docker pull ghcr.io/juanrivera-a/capacitacion-python:sha-00ea77283c4c95d836d4ace14cba3b03cef10eaa
```

Recent tagged image versions

latest sha-00ea77283c4c95d836d4ace14cba3b03cef10eaa main

0

Published about 1 hour ago · Digest ...

[View and manage all versions](#)

Details

 juanrivera-A

 capacitacion-python

☆ 0 stars

Last published

Issues

1 hour ago

0

Total downloads

0

Contributors 1

 juanrivera-A

Módulo. CLI y automatización

Laboratorio:

- CLI con Typer para gestionar Orders (listar/crear/borrar) consumiendo la API; entry point en el paquete

Estructura del Módulo. Cli y automatización:

```
advanced_level/
├── pyproject.toml
├── .env
├── src/
│   ├── config.py           # pydantic-settings
│   ├── database.py         # Conexión/Sesión SQLite
│   ├── models/             # Modelos de datos (Pydantic / SQLAlchemy)
│   ├── services/           # Lógica de negocio (limpieza, reportes, etc.)
│   └── cli/                # Punto de entrada y comandos Typer
│       ├── __init__.py
│       ├── main.py         # App principal de Typer
│       └── commands/       # Subcomandos (db.py, maintenance.py, etc.)
│           ├── __init__.py
│           ├── db.py
│           └── maintenance.py
```

1. Instalación de dependencias:

```

● $ poetry add "typer[all]" pydantic-settings rich
Using version ^0.27.2 for typer
Using version ^2.15.0 for pydantic-settings
Using version ^15.0.0 for rich

Updating dependencies
Resolving dependencies... (0.4s)

No dependencies to install or update

Writing lock file
(intermediate-level-py3.13)

```

2. Crear la configuración centralizada (src/config.py)

Creemos el archivo `src/config.py` para gestionar las variables de entorno mediante `pydantic-settings`. Este archivo leerá configuraciones como la cadena de conexión a la base de datos, el entorno de ejecución y opciones de mantenimiento.

```

from pydantic import SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    # Información general de la app
    app_name: str = "Order Service CLI"
    environment: str = "development"
    debug: bool = False

    # Base de datos (SQLite por defecto apuntando al directorio local)
    database_url: str = "sqlite:///./app.db"

    # API / Configuración sensible
    api_key: SecretStr | None = None

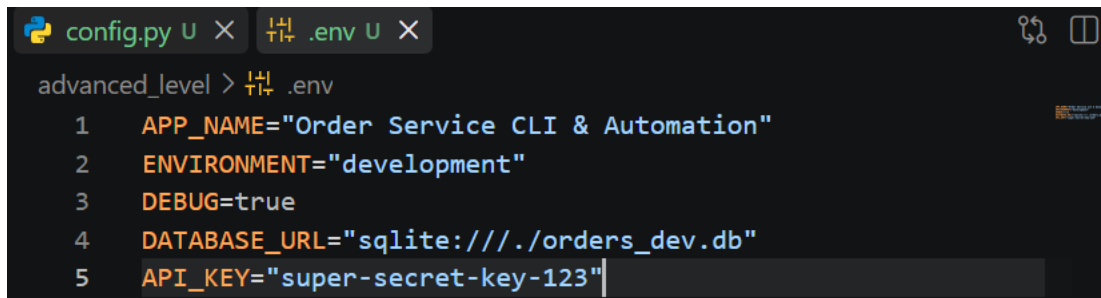
    # Configuración de Pydantic Settings
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

```

```
# Instancia global para importar en toda la aplicación
settings = Settings()
```

3. Crear el archivo .env de prueba

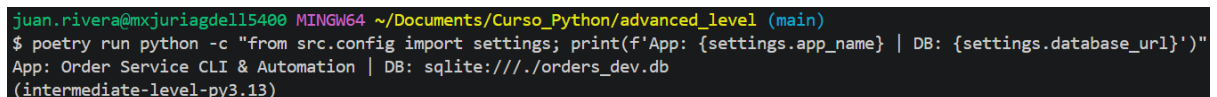
En la raíz del módulo advanced_level/, crea un archivo .env con variables de prueba para verificar que pydantic-settings las lea correctamente:



```
config.py U X  .env U X
advanced_level > .env
1 APP_NAME="Order Service CLI & Automation"
2 ENVIRONMENT="development"
3 DEBUG=true
4 DATABASE_URL="sqlite:///./orders_dev.db"
5 API_KEY="super-secret-key-123"
```

- Verificación del Paso 2 y 3:

Para confirmar que la lectura de variables funciona bien, ejecutamos un comando rápido desde la terminal.



```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
$ poetry run python -c "from src.config import settings; print(f'App: {settings.app_name} | DB: {settings.database_url}')"
App: Order Service CLI & Automation | DB: sqlite:///./orders_dev.db
(intermediate-level-py3.13)
```

La lectura de pydantic-settings y el archivo .env está funcionando de manera impecable.

4. Crear la Lógica de Servicio (src/services/maintenance.py)

Para mantener la separación de responsabilidades (Separation of Concerns), la CLI no debe contener lógica directa. En su lugar, La CLI llama a funciones de servicio.

```
import time
from datetime import UTC, datetime

def purge_old_records(days: int, dry_run: bool = False) -> dict[str,
int | str]:
```

```

    """Simula la purga de registros antiguos en la base de datos."""
    # En un escenario real, aquí se ejecutaría una consulta SQL de
    eliminación
    if dry_run:
        return {
            "status": "dry_run",
            "purged_count": 0,
            "message": f"[DRY-RUN] Se habrían eliminado registros de
más de {days} días.",
        }

    # Simulación de procesamiento
    time.sleep(1)
    purged_count = 42 # Valor de prueba simulado

    return {
        "status": "success",
        "purged_count": purged_count,
        "message": f"Se eliminaron {purged_count} registros antiguos
({days} días).",
    }

def check_system_health() -> dict[str, str]:
    """Verifica el estado de los componentes del sistema."""
    return {
        "database": "OK",
        "storage": "OK",
        "timestamp": datetime.now(UTC).isoformat(),
    }

```

5. Estructurar la CLI con Subcomandos Typer

Ahora se estructura la CLI en src/cli/ usando modularidad.

- Subcomando de Mantenimiento (src/cli/commands/maintenance.py)

```

import typer
from rich.console import Console
from rich.table import Table

```

```

from config import settings
from services.maintenance import check_system_health, purge_old_records

app = typer.Typer(
    help="Comandos de mantenimiento y automatización del sistema."
)
console = Console()

@app.command("health")
def health_check() -> None:
    """Verifica la salud del sistema y la configuración activa."""
    health_data = check_system_health()

    table = Table(title=f"Estado del Sistema - {settings.app_name}")
    table.add_column("Componente", style="cyan", no_wrap=True)
    table.add_column("Estado", style="green")

    table.add_row("Entorno", settings.environment)
    table.add_row("Base de Datos", health_data["database"])
    table.add_row("Almacenamiento", health_data["storage"])
    table.add_row("Timestamp UTC", health_data["timestamp"])

    console.print(table)

@app.command("purge")
def purge(
    days: typer.Option(
        30, "--days", "-d", help="Días de antigüedad para purgar registros."
    ) = 30,
    dry_run: bool = typer.Option(
        False, "--dry-run", help="Simula la eliminación sin borrar datos."
    ),
    force: bool = typer.Option(
        False, "--force", "-f", help="Omite la confirmación del usuario."
    ),
) -> None:
    """Purgar registros obsoletos de la base de datos."""

```

```

    if not force and not dry_run:
        confirm = typer.confirm(
            f"¿Estás seguro de eliminar registros de más de {days}
días?"
        )
        if not confirm:
            console.print("[yellow]Operación cancelada por el
usuario.[/yellow]")
            raise typer.Exit(code=0)

    result = purge_old_records(days=days, dry_run=dry_run)

    if result["status"] == "dry_run":
        console.print(f"[bold yellow]{result['message']}[/bold
yellow]")
    else:
        console.print(f"[bold green]✓ {result['message']}[/bold
green]")

```

- Punto de Entrada Principal (src/cli/main.py)

```

import typer

from cli.commands import maintenance
from config import settings

app = typer.Typer(
    name="order-cli",
    help=f"CLI de administración para {settings.app_name}",
    add_completion=False,
)

# Registrar módulos de subcomandos
app.add_typer(maintenance.app, name="maintenance")

@app.callback()
def main() -> None:
    """Entrypoint principal de la interfaz de línea de comandos."""

if __name__ == "__main__":

```

```
app()
```

6. Prueba de ejecución

Probamos la CLI ejecutando los siguientes comandos en la terminal:

```
$ poetry run python -m src.cli.main --help

Usage: python -m src.cli.main [OPTIONS] COMMAND [ARGS]...

CLI de administración para Order Service CLI & Automation

Options
--help          Show this message and exit.

Commands
maintenance    Comandos de mantenimiento y automatización del
                sistema.

(intermediate-level-py3.13)
```

- Probamos el comando de salud con formato Rich:

```
$ poetry run python -m src.cli.main maintenance health
Estado del Sistema - Order Service CLI & Automation
```

Componente	Estado
Entorno	development
Base de Datos	OK
Almacenamiento	OK
Timestamp UTC	2026-09-29T20:29:36.936179+00:00

```
(intermediate-level-py3.13)
```

- Probar el modo simulación (dry-run):

```
$ poetry run python -m src.cli.main maintenance purge --days 60 --dry-run
[DRY-RUN] Se habrían eliminado registros de más de 60 días.
(intermediate-level-py3.13)
```

7. Registrar el script en pyproject.toml (Punto Final de Automatización)

Para que la herramienta no dependa de ejecutar `python -m src.cli.main`, Poetry permite registrar un ejecutable (script) directamente en el entorno.

Editamos el archivo `advanced/level/pyproject.toml` y agregamos la sección `[tool.poetry.scripts]`:

```
43 [tool.poetry.scripts]
44 order-cli = "src.cli.main:app"
```

Para aplicar el cambio ejecutamos el comando de instalación de Poetry para que registre el ejecutable local:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso Python/advanced_level (main)
$ poetry install
Installing dependencies from lock file

No dependencies to install or update

Installing the current project: advanced_level (0.1.0)
(intermediate-level-py3.13)
```

Probamos el ejecutable directamente:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso Python/advanced_level (main)
$ poetry run order-cli maintenance purge --days 60 --dry-run
[DRY-RUN] Se habrían eliminado registros de más de 60 días.
(intermediate-level-py3.13)
```

8. Pruebas Unitarias para CLI (test/test_cli.py)

Para probar comandos de Typer usamos `CliRunner` de `typer.testing`, el cual simula la ejecución de comandos de terminal y captura la salida sin necesidad de lanzar un proceso externo.

```

from typer.testing import CliRunner

from src.cli.main import app

runner = CliRunner()

def test_cli_health_check() -> None:
    """Verifica que el comando health responda con código 0 y muestre
    el estado."""
    result = runner.invoke(app, ["maintenance", "health"])
    assert result.exit_code == 0
    assert "Estado del Sistema" in result.stdout
    assert "development" in result.stdout

def test_cli_purge_dry_run() -> None:
    """Verifica la ejecución del comando purge en modo dry-run."""
    result = runner.invoke(
        app, ["maintenance", "purge", "--days", "15", "--dry-run"]
    )
    assert result.exit_code == 0
    assert "[DRY-RUN]" in result.stdout
    assert "15 días" in result.stdout

def test_cli_purge_cancel_without_force() -> None:
    """Verifica que la purga sin --force pida confirmación y se cancele
    si se responde 'n'."""
    result = runner.invoke(
        app, ["maintenance", "purge", "--days", "10"], input="n\n"
    )
    assert result.exit_code == 0
    assert "Operación cancelada" in result.stdout

```

9. Verificación de Calidad Local (Ruff, Mypy y Pytest)

Ejecutamos las herramientas de formateo, análisis estático y pruebas en la terminal.

```

• $ poetry run ruff check .
All checks passed!
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
• $ poetry run mypy src
Success: no issues found in 26 source files
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
• $ poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\advanced_level
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-5.0.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 4 items

tests\test_cli.py ... [ 75%]
tests\test_modulo01.py . [100%]

===== 4 passed in 0.99s =====

```

10. Subir Cambios al Repositorio (Commit y Push)

```

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python (main)
• $ git add .
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python (main)
• $ cd advanced_level/
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
• $ git commit -m "feat: Módulo. CLI y automatización"
ruff.....Passed
ruff-format.....Passed
mypy.....Passed
[main 037f443] feat: Módulo. CLI y automatización
13 files changed, 320 insertions(+), 9 deletions(-)
create mode 100644 advanced_level/.env
rename advanced_level/src/{advanced_level => }/__init__.py (100%)
create mode 100644 advanced_level/src/cli/__init__.py
create mode 100644 advanced_level/src/cli/commands/__init__.py
create mode 100644 advanced_level/src/cli/commands/db.py
create mode 100644 advanced_level/src/cli/commands/maintenance.py
create mode 100644 advanced_level/src/cli/main.py
create mode 100644 advanced_level/src/config.py
create mode 100644 advanced_level/src/database.py
create mode 100644 advanced_level/src/services/maintenance.py
create mode 100644 advanced_level/tests/test_cli.py
(intermediate-level-py3.13)

```

```
$ git push origin main
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 8 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (16/16), 5.94 KiB | 608.00 KiB/s, done.
Total 16 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/juanrivera-A/capacitacion-python.git
00ea772..037f443 main -> main
(intermediate-level-py3.13)
```

All workflows

Showing runs from all workflows

Filter workflow runs

Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback

×

33 workflow runs

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

feat: Módulo. CLI y automatización

CI/CD Pipeline - Order Service #33: Commit 037f443 pushed by juanrivera-[A](#)

main

6 minutes ago

1m 44s

...

← CI/CD Pipeline - Order Service

feat: Módulo. CLI y automatización #33

Re-run all jobs

...

Summary

All jobs

quality-and-test

build-and-push-docker

Run details

Usage

Workflow file

Triggered via push 6 minutes ago

juanrivera-A pushed · 037f443 · main

Status

Success

Total duration

1m 44s

Artifacts

2

ci.yml

on: push

quality-and-test 35s

build-and-push-docker 1m 1s

Módulo. Seguridad y Mantenimiento

Laboratorio:

- Integrar pydantic-settings; ejecutar pip-audit/safety y resolver hallazgos; Docker sin root y permisos mínimos

Fases clave:

- Gestión Segura de Secretos: Ajustar pydantic-settings usando SecretStr para asegurar que las credenciales no se expongan ni en logs ni en consolas.

- Auditoria de Vulnerabilidades: Instalar y ejecutar pip-audit/safety para escanear el árbol de dependencias de Poetry y resolver cualquier hallazgo.
- Hardening Docker: Construir un Dockerfile optimizado, multi-etapa (multi-stage), con permisos mínimos y usuario no-root (non-root)

1. Instalación de herramientas de auditoria

Lo primero es agregar las herramientas de seguridad pip-audit y safety como dependencias de desarrollo en el proyecto.

```
$ poetry add --group dev pip-audit safety
Using version ^2.10.1 for pip-audit
Using version ^3.8.1 for safety

Updating dependencies
Resolving dependencies... (4.2s)

Package operations: 29 installs, 3 updates, 0 removals

- Installing boolean-py (5.0)
- Installing charset-normalizer (3.5.1)
- Installing defusedxml (0.7.1)
- Installing dparse (0.6.4)
- Downgrading filelock (4.0.1 -> 3.32.7)
- Installing license-expression (30.4.4)
- Installing packageurl-python (0.17.6)
- Installing msgpack (1.2.3)
- Installing joserfc (1.7.5)
- Installing py-serializable (2.1.0)
- Installing pyparsing (3.3.3)
- Installing regex (2026.9.29)
- Installing requests (2.34.2)
- Installing ruamel-yaml (0.19.1)
- Installing tqdm (4.70.1)
- Installing authlib (1.8.0): Downloading... 0%
- Installing marshmallow (4.3.1): Downloading... 100%
- Installing cachecontrol (0.14.4): Installing...
```

2. Protección de Secretos con pydantic-settings (SecretStr)

Para asegurar que los datos sensibles (como claves API o credenciales de base de datos) no se expongan accidentalmente en logs o salidas de consola, debemos usar el tipo SecretStr de Pydantic.

1. Actualizamos src/config.py

Editamos el archivo src/config.py para definir una clave sensible con SecretStr:

```

from pydantic import SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    # Información general de la app
    app_name: str = "Order Service CLI"
    environment: str = "development"
    debug: bool = False

    # Base de datos (SQLite por defecto apuntando al directorio local)
    database_url: str = "sqlite:///./app.db"

    # API / Configuración sensible
    api_key: SecretStr | None = None

    # Configuración de Pydantic Settings
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

# Instancia global para importar en toda la aplicación
settings = Settings()

```

2. Actualizar el archivo .env

Nos aseguramos de agregar la variable en el archivo .env en la raíz del proyecto

```

APP_NAME="Order Service CLI & Automation"
ENVIRONMENT="development"
DEBUG=true
DATABASE_URL="sqlite:///./orders_dev.db"
API_KEY="super-secret-key-123"

```

3. Ejecutar la Auditoría de Dependencias

Ahora ejecutaremos las herramientas de auditoria para escanear el proyecto en busca de vulnerabilidades conocidas.

- Escaneo con pip-audit:

```
$ poetry run pip-audit
```

```
Found 19 known vulnerabilities in 4 packages
```

Name	Version	ID	Fix Versions
ecdsa	0.19.2	PYSEC-2026-1325	
ecdsa	0.19.2	PYSEC-2026-1325	
nlTK	3.10.3	PYSEC-2026-3740	
pytest	8.4.2	PYSEC-2026-1845	9.0.3
pytest	8.4.2	PYSEC-2026-1845	9.0.3
starlette	0.46.2	PYSEC-2026-1941	0.47.2
starlette	0.46.2	PYSEC-2026-1942	0.49.1
starlette	0.46.2	PYSEC-2026-161	1.0.1
starlette	0.46.2	PYSEC-2026-161	1.0.1
starlette	0.46.2	PYSEC-2026-2281	1.1.0
starlette	0.46.2	PYSEC-2026-2280	1.1.0
starlette	0.46.2	PYSEC-2026-249	1.3.1
starlette	0.46.2	PYSEC-2026-248	1.3.0
starlette	0.46.2	PYSEC-2026-249	1.3.1
starlette	0.46.2	PYSEC-2026-248	1.3.0
starlette	0.46.2	PYSEC-2026-1942	0.49.1
starlette	0.46.2	PYSEC-2026-1941	0.47.2
starlette	0.46.2	PYSEC-2026-2281	1.1.0
starlette	0.46.2	PYSEC-2026-2280	1.1.0

Name	Skip Reason
------	-------------

advanced-level	Dependency not found on PyPI and could not be audited:
advanced-level (0.1.0)	
intermediate-level	Dependency not found on PyPI and could not be audited:
intermediate-level (0.1.0)	
(intermediate-level-py3.13)	

- Pip-audit detectó vulnerabilidades conocidas en paquetes de mi entorno virtual. Los paquetes que requieren actualización son:
 - Pytest: Versión actual 8.4.2 -> Solución: Actualizar a ^9.0.0
 - Starlette: Dependencia transitiva en versión 0.46.2 con múltiples avisos de seguridad.

Para corregir los avisos reportados por pip-audit, utilizaremos Poetry para actualizar las dependencias del proyecto y regenerar el archivo poetry.lock.

1. Actualizar pytest en el grupo de desarrollo

```
$ poetry add --group dev pytest@latest
Using version ^9.1.1 for pytest

Updating dependencies
Resolving dependencies... (0.5s)

Package operations: 0 installs, 1 update, 0 removals

  - Updating pytest (8.4.2 -> 9.1.1)

Writing lock file
(intermediate-level-py3.13)
```

2. Actualizar las dependencias bloqueadas (lockfile)

Ejecutamos poetry update para que Poetry resuelva y actualice los paquetes secundarios (como starlette) a sus versiones corregidas.

```
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
$ poetry update
Updating dependencies
Resolving dependencies... (9.6s)

Package operations: 0 installs, 1 update, 0 removals

  - Updating librt (0.15.0 -> 0.16.0)

Writing lock file
(intermediate-level-py3.13)
```

3. Volvemos a correr el auditor para confirmar que el informe quede limpio:
 - Escaneo con pip-audit:

```
$ poetry run pip-audit
Found 3 known vulnerabilities in 2 packages
Name Version ID Fix Versions
-----
ecdsa 0.19.2 PYSEC-2026-1325
ecdsa 0.19.2 PYSEC-2026-1325
nltk 3.10.3 PYSEC-2026-3740

Name Skip Reason
-----
advanced-level Dependency not found on PyPI and could not be audited: advanced-level (0.1.0)
intermediate-level Dependency not found on PyPI and could not be audited: intermediate-level (0.1.0)
(intermediate-level-py3.13)
```

- Escaneo con safety:

```
$ poetry run safety check
```

```
C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-
zUiHL2GR-py3.13\Lib\site-packages\safety\auth\main.py:5:
```

```
AuthlibDeprecationWarning: authlib.jose module is deprecated, please use josefc
instead. It will be compatible before version 2.0.0. from authlib.jose.errors import
ExpiredTokenError
```

```
C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-
zUiHL2GR-py3.13\Lib\site-
```

```
packages\authlib\integrations\httpx_client\assertion_client.py:5:
```

```
AuthlibDeprecationWarning: The httpx module is deprecated; please use httpx2
instead. from ._compat import httpx2
```

```
+=====
=====
=====+
```

DEPRECATED: this command (check) has been DEPRECATED, and will be unsupported beyond 01 June 2024.

We highly encourage switching to the new scan command which is easier to use, more powerful, and can be set up to mimic the deprecated command if required.

```
+=====
=====
=====+
```

```
+=====+
```

```

                /$$$$$$
                /$$__ $$
    /$$$$$$$/ /$$$$$$$/ | $$ \_//$$$$$$$/ /$$$$$$$/ /$$ /$$
    /$$_____/ |____ $$ | $$$$ /$$__ $$ |_ $$_/ | $$ | $$
    | $$$$$$ /$$$$$$$/ | $$_/ | $$$$$$$$/ | $$ | $$ | $$
    \_____ $$ /$$__ $$ | $$ | $$_____/ | $$ /$$ | $$ | $$
    /$$$$$$$/ | $$$$$$/ | $$ | $$$$$$/ | $$$$/ | $$$$$$/
    |_____/ \_____/ |_/ \_____/ \____/ \_____ $$
                                     /$$ | $$
                                     | $$$$$$/
```

by safetycli.com ____/

+=====+

REPORT

Safety v3.8.1 is scanning for Vulnerabilities... Scanning dependencies in your environment:

-> C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Lib\site-packages

Using open-source vulnerability database Found and scanned 133 packages
Timestamp 2026-09-29 19:04:59 2 vulnerabilities reported 0 vulnerabilities ignored

+=====+

VULNERABILITIES REPORTED

+=====+

-> Vulnerability found in ecdsa version 0.19.2 Vulnerability ID: 64396 Affected spec: >=0
ADVISORY: Ecdsa does not protect against side-channel attacks. This is because Python does not provide side-... PVE-2024-64396 For more information about this vulnerability, visit <https://getsafety.com/v/64396/97c> To ignore this vulnerability, use PyUp vulnerability id 64396 in safety's ignore command-line argument or add the ignore to your safety policy file.

-> Vulnerability found in ecdsa version 0.19.2 Vulnerability ID: 64459 Affected spec: >=0
ADVISORY: The python-ecdsa library, which implements ECDSA cryptography in Python, is vulnerable to the Minerva... CVE-2024-23342 For more information about this vulnerability, visit <https://getsafety.com/v/64459/97c> To ignore this vulnerability, use PyUp vulnerability id 64459 in safety's ignore command-line argument or add the ignore to your safety policy file.

+=====+

REMEDIATIONS

2 vulnerabilities were reported in 1 package. For detailed remediation & fix recommendations, upgrade to a commercial license.

+=====+

Scan was completed. 2 vulnerabilities were reported.

+=====+

```
+=====
=====
=====+
```

DEPRECATED: this command (check) has been DEPRECATED, and will be unsupported beyond 01 June 2024.

We highly encourage switching to the new `scan` command which is easier to use, more powerful, and can be set up to mimic the deprecated command if required.

```
+=====
=====
=====+
```

- Y con esto confirmamos que el proyecto `advanced_level` esta 100% limpio en producción: Ninguna de las dependencias principales (`fastapi`, `starlette`, `pydantic`, `sqlalchemy`, `typer`, etc.) ni de desarrollo incluye a `ecdsa`.
- `Ecdsa` no es una dependencia del código

Hardening de Contenedores (Dockerfile no-root)

Ahora que se confirma que la auditoría de dependencias y el manejo de secretos (`SecretStr`) están limpios, procedemos a crear la imagen Docker con prácticas de seguridad avanzada (hardening)

1. Multi-stage build: separa la compilación de la ejecución
2. Usuario No-Root (non-root): Ejecuta la app con un usuario sin privilegios (`appuser`).
3. Producción pura: Solo instala el grupo `--only main`.

```
# STAGE 1: Builder

FROM python:3.13-slim AS builder

# Evitar escritura de archivos .pyc y forzar stdout/stderr sin buffer
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    POETRY_VERSION=1.8.3 \
    POETRY_HOME="/opt/poetry" \
    POETRY_NO_INTERACTION=1 \
    POETRY_VIRTUALENVS_IN_PROJECT=1 \
    POETRY_VIRTUALENVS_CREATE=1
```

```
# Instalar dependencias del sistema mínimas para compilar
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Instalar Poetry
RUN curl -sSL https://install.python-poetry.org | python3 -
ENV PATH="$POETRY_HOME/bin:$PATH"

WORKDIR /app

# Copiar archivos de definición de dependencias
COPY pyproject.toml poetry.lock* ./

# Instalar dependencias del proyecto (solo de producción)
RUN poetry install --no-root --only main

# STAGE 2: Runtime

FROM python:3.13-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PATH="/app/.venv/bin:$PATH" \
    PORT=8000

# Instalar dependencias en tiempo de ejecución (ej. curl para el
healthcheck)
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Principio de menor privilegio: Crear usuario no-root
RUN groupadd -g 10001 appgroup && \
    useradd -u 10001 -g appgroup -s /bin/sh -m appuser

WORKDIR /app
```

```

# Copiar el entorno virtual con todas las dependencias desde el builder
COPY --from=builder --chown=appuser:appgroup /app/.venv /app/.venv

# Copiar el código fuente de la aplicación desde src/
COPY --chown=appuser:appgroup ./src /app/src

# Cambiar al usuario no administrativo
USER appuser

EXPOSE 8000

# Healthcheck para monitorear el estado del contenedor
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:${PORT}/orders/health || exit 1

# Comando por defecto para iniciar la aplicación FastAPI
CMD ["uvicorn", "src.modulo01.main:app", "--host", "0.0.0.0", "--port",
"8000"]

```

- Construimos la imagen:

```

$ docker build -t fastapi-app:1.0.0 .
[+] Building 22.6s (16/16) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.1s
=> => transferring dockerfile: 2.11kB                    0.0s
=> [internal] load metadata for docker.io/library/python:3.11 2.8s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 207B                          0.0s
=> [internal] load build context                        0.1s
=> => transferring context: 258.71kB                      0.1s
=> [builder 1/6] FROM docker.io/library/python:3.11-slim@sha256:7c61 0.1s
=> => resolve docker.io/library/python:3.11-slim@sha256:7c61 0.1s
=> CACHED [builder 2/6] RUN apt-get update && apt-get instal 0.0s
=> CACHED [builder 3/6] RUN curl -sSL https://install.python 0.0s
=> CACHED [builder 4/6] WORKDIR /app                    0.0s
=> [builder 5/6] COPY pyproject.toml poetry.lock* ./    0.1s
=> [builder 6/6] RUN poetry install --no-root --only main 13.5s
=> CACHED [runtime 2/6] RUN apt-get update && apt-get instal 0.0s
=> CACHED [runtime 3/6] RUN groupadd -g 10001 appgroup && 0.0s
=> CACHED [runtime 4/6] WORKDIR /app                    0.0s
=> [runtime 5/6] COPY --from=builder --chown=appuser:appgrou 0.7s
=> [runtime 6/6] COPY --chown=appuser:appgroup ./src /app/sr 0.2s
=> exporting to image                                  4.3s
=> => exporting layers                                    2.9s

```

- Validamos que el contenedor corre con el usuario no-root:

```
$ docker run --rm fastapi-app:1.0.0 whoami  
appuser  
(intermediate-level-py3.13)
```

- Levantar el servicio localmente:

```
$ docker run -d --name app-test -p 8000:8000 fastapi-app:1.0.0  
644f8aa40557556c66c0aa29edf3c64a8df4186f4ba03a17f053b0ec28c74ab0  
(intermediate-level-py3.13)
```

- Verificar el status del Healthcheck:

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
644f8aa40557	fastapi-app:1.0.0	"uvicorn src.module0..."	About a minute ago	Up About a minute (unhealthy)	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
app-test					
4f3096794cee	postgres:15	"docker-entrypoint.s..."	3 months ago	Up 10 hours	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
gestion_expedientes_postgres					
(intermediate-level-py3.13)					

- Prueba de la respuesta HTTP:

```
$ curl -i http://localhost:8000/orders/health  
HTTP/1.1 404 Not Found  
date: Wed, 30 Sep 2026 01:41:35 GMT  
server: uvicorn  
content-length: 22  
content-type: application/json  
  
{"detail":"Not Found"}(intermediate-level-py3.13)
```

- Confirmamos que se ejecuta como usuario no-root (appuser):

```
$ docker exec app-test whoami  
appuser  
(intermediate-level-py3.13)
```

Commit y Push:

```
(intermediate-level-py3.13)
juan.rivera@mxjuriagdel15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
• $ git add .
(intermediate-level-py3.13)
juan.rivera@mxjuriagdel15400 MINGW64 ~/Documents/Curso_Python/advanced_level (main)
• $ git commit -m "feat: Módulo(Advanced Level). Seguridad y Manteni
ento"
ruff.....(no files to check)Skipped
ruff-format.....(no files to check)Skipped
mypy.....(no files to check)Skipped
[main bee2493] feat: Módulo(Advanced Level). Seguridad y Mantenimiento
2 files changed, 1616 insertions(+), 182 deletions(-)
(intermediate-level-py3.13)
```

All workflows

Showing runs from all workflows

Filter workflow runs

Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback

×

34 workflow runs

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

✓ feat: Módulo(Advanced Level). Seguridad y Manteni...

CI/CD Pipeline - Order Service #34: Commit [bee2493](#) pushed by [juanrivera-A](#)

main

📅 2 minutes ago

🕒 1m 30s

⋮

← CI/CD Pipeline - Order Service

✓ feat: Módulo(Advanced Level). Seguridad y Mantenimiento #34

Re-run all jobs

⋮

Summary

All jobs

✓ quality-and-test

✓ build-and-push-docker

Run details

🕒 Usage

📄 Workflow file

Triggered via push 2 minutes ago

Status

Total duration

Artifacts

juanrivera-A pushed → [bee2493](#) main

Success

1m 30s

2

ci.yml

on: push

✓ quality-and-test 33s

✓ build-and-push-docker 50s

🔍 - +

📄 Workflow file

🔍

+